

Módulo 18: CLI y automatización

Contenidos clave

CLI: (Command Line Interface) Aplicación que se puede manejar desde la terminal, recibe instrucciones desde la terminal.

argparse, click y Typer:

- `argparse`: es una librería incluida en Python para crear CLIs. Permite comandos, opciones, tipos, valores por defecto, ayuda, etc.. La ventaja es que no se necesita instalar. Desventaja: para CLIs grandes puede terminar siendo bastante código.
- `Click`: (framework) es una librería externa que facilita la creación de CLIs.
- `Typer`: está construido sobre `Click` y utiliza los `type hints` de Python para definir los comandos.

Una CLI normalmente tiene varios comandos, cada uno ejecuta una operación diferente.

Configuración por variables de entorno: La CLI necesita saber dónde está la API. Se pueden usar variables de entorno para cosas sensibles, así se evita poner secretos directamente en el código. En producción los valores sensibles los administra el entorno de ejecución o un gestor de secretos .

Scripts de mantenimiento: es un pequeño programa que automatiza tareas administrativas o repetitivas.

```
scripts/  
├─ clean_database.py  
├─ generate_report.py  
├─ backup.py  
└─ migrate.py
```

Objetivos

- Construir CLIs productivas y mantenibles.

Implementación de CLI con `Typer` que aporta:

- Generación automática de ayuda
- Validación de tipos
- Opciones y argumentos documentados
- Validación de cantidades y parámetros
- Mensajes claros para el usuario

- Integrar automatizaciones del proyecto.

La CLI se conectó con la API de Orders mediante el cliente definido en:

```
21     def list_orders(self) -> list[dict[str, Any]]:
22         response = httpx.get(
23             f"{self.base_url}/orders/",
24             headers=self._headers(),
25             timeout=self.timeout,
26         )
27         response.raise_for_status()
28         return response.json()
```

Los comandos de la CLI no manejan directamente detalles de HTTP; delegan las operaciones a OrdersClient.

La CLI también se integró como entry point del paquete en pyproject.toml:

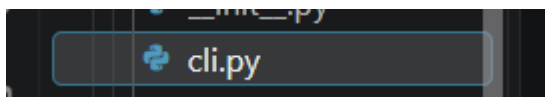
```
117 [project.scripts]
118 orders = "labs.day18_cli.cli:app"
```

Para poder ejecutar poetry run orders --help sin tener que invocar directamente el archivo Python.

Laboratorio

- CLI con Typer para gestionar Orders (listar/crear/borrar) consumiendo la API:

La aplicación se implementó en:



Con los comandos:

```
Commands
list
create
delete
```

- Listar órdenes: list.

```
21 @app.command("list")
22 def list_orders(
23     token: str | None = typer.Option(
24         default=None,
25         envvar="ORDERS_API_TOKEN",
26         help="Token JWT.",
27     ),
28 ) -> None:
29     client = get_client(token)
30     orders = client.list_orders()
```

Este comando consume GET /orders/ y muestra:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run orders list
#1 customer=1 status=PENDING
#2 customer=1 status=PENDING
```

- Crear una orden: create.

```
45 @app.command()
46 def create(
47     status: str = typer.Option(
48         default="PENDING",
49         help="Estado de la orden.",
50     ),
51     product_name: str = typer.Option(
52         ...,
53         prompt="Nombre del producto",
54     ),
55     quantity: int = typer.Option(
56         ...,
57         min=1,
58         prompt="Cantidad",
59     ),
60     unit_price: float = typer.Option(
61         ...,
62         min=0.01,
63         prompt="Precio unitario",
64     ),
65     token: str | None = typer.Option(
66         default=None,
67         envvar="ORDERS_API_TOKEN",
68         help="Token JWT.",
69     ),
70 ) -> None:
71     client = get_client(token)
```

Este comando consume: POST /orders/ y envía:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run orders create
Nombre del producto: luces led
Cantidad: 3
Precio unitario: 124.48
Orden creada: #3
```

- Borrar una orden: delete.

```
87 @app.command()
88 def delete(
89     order_id: int = typer.Argument(
90         ...,
91         min=1,
92     ),
93     token: str | None = typer.Option(
94         default=None,
95         envvar="ORDERS_API_TOKEN",
96         help="Token JWT.",
97     ),
98 ) -> None:
99     client = get_client(token)
100     client.delete_order(order_id)
```

Este comando consume: DELETE / orders/{order_id} y muestra:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\
python-mastery-capstone> poetry run orders delete 3
Orden eliminada: #3
```